Visualizing genus zero dessins

michael musty pydata BTV July 30, 2024

intro

background

- number theory, computation

past

- postdoc (ICERM)
- mathematician (CRREL)
  - computer vision, mixed models

current

- data scientist (Faraday)
  - product implementation
  - modeling / API infrastructure
  - (among other things)

outline

1. motivating example
2. some background
3. or-tools
4. closing

repositories

https://github.com/faradayio/RepAssignment

repositories

https://github.com/faradayio/RepAssignment

- https://github.com/dbcrossbar/
- https://github.com/faradayio/cage
- https://github.com/faradayio/joinery

sales rep assignment

sales rep assignment

- A solar company Acme Inc. has inbound residential customers (leads l_i)

sales rep assignment

- A solar company Acme Inc. has inbound residential customers (leads l_i)
- Each lead schedules a day to have an Acme sales rep r_j visit the lead's residence

sales rep assignment

- A solar company Acme Inc. has inbound residential customers (leads l_i)
- Each lead schedules a day to have an Acme sales rep r_j visit the lead's residence
- Acme has the ability to determine the expected profit e_{ij} from having r_j visit l_i

sales rep assignment

- A solar company Acme Inc. has inbound residential customers (leads l_i)
- Each lead schedules a day to have an Acme sales rep r_j visit the lead's residence
- Acme has the ability to determine the expected profit e_{ij} from having r_j visit l_i
- **objective**: Determine which rep to assign to each lead to maximize the sum of expected profits each day

sales rep assignment

- A solar company Acme Inc. has inbound residential customers (leads l_i)
- Each lead schedules a day to have an Acme sales rep r_j visit the lead's residence
- Acme has the ability to determine the expected profit e_{ij} from having r_j visit l_i
- **objective**: Determine which rep to assign to each lead to maximize the sum of expected profits each day
- subject to the **constraints**:
  - every lead get assigned exactly 1 rep
  - every rep visits at most 1 lead per day

setup

$$
\begin{aligned}
l_i &= \text{lead } i, \quad i \in \{0, \ldots, L-1\} \\
r_j &= \text{rep } j, \quad j \in \{0, \ldots, R-1\} \\
e_{ij} &= \text{expected profit of assigning } l_i \text{ to } r_j \\
x_{ij} &= \begin{cases} 1 & \text{if } l_i \text{ is assigned to } r_j \\ 0 & \text{otherwise} \end{cases}
\end{aligned}
$$

objective and constraints

objective and constraints

- **objective**: choose x_{ij} to maximize

$$\sum_{i,j} e_{ij} x_{ij}$$

objective and constraints

- **objective**: choose x_{ij} to maximize

$$\sum_{i,j} e_{ij} x_{ij}$$

- subject to the **constraints**:
  - $\sum_{j=0}^{R-1} x_{ij} = 1$, for every l_i

objective and constraints

- **objective**: choose x_{ij} to maximize

$$\sum_{i,j} e_{ij} x_{ij}$$

- subject to the **constraints**:
  - $\sum_{j=0}^{R-1} x_{ij} = 1$, for every l_i
    i.e. *every lead gets assigned exactly 1 rep*

objective and constraints

- **objective**: choose x_{ij} to maximize

$$\sum_{i,j} e_{ij} x_{ij}$$

- subject to the **constraints**:
  - $\sum_{j=0}^{R-1} x_{ij} = 1$, for every l_i
    i.e. *every lead gets assigned exactly 1 rep*
  - $\sum_{i=0}^{L-1} x_{ij} \leq 1$, for every r_j

objective and constraints

- **objective**: choose x_{ij} to maximize

$$\sum_{i,j} e_{ij} x_{ij}$$

- subject to the **constraints**:
  - $\sum_{j=0}^{R-1} x_{ij} = 1$, for every l_i
    i.e. *every lead gets assigned exactly 1 rep*
  - $\sum_{i=0}^{L-1} x_{ij} \leq 1$, for every r_j
    i.e. *every rep visits at most 1 lead per day*

example $L = 2, R = 3$

example $L = 2, R = 3$

- Given expected profits

$$(e_{ij}) = \begin{pmatrix} 4 & 5 & 2 \\ 1 & 3 & 1 \end{pmatrix}, \quad \text{i.e. } e_{11} = 3, e_{01} = 5, \ldots$$

example $L = 2, R = 3$

- Given expected profits

$$(e_{ij}) = \begin{pmatrix} 4 & 5 & 2 \\ 1 & 3 & 1 \end{pmatrix}, \quad \text{i.e. } e_{11} = 3, e_{01} = 5, \ldots$$

- maximize

$$\sum_{i,j} e_{ij}x_{ij} = e_{00}x_{00} + e_{01}x_{01} + e_{02}x_{02} + e_{10}x_{10} + e_{11}x_{11} + e_{12}x_{12}$$

example $L = 2, R = 3$

- Given expected profits

$$(e_{ij}) = \begin{pmatrix} 4 & 5 & 2 \\ 1 & 3 & 1 \end{pmatrix}, \quad \text{i.e. } e_{11} = 3, e_{01} = 5, \ldots$$

- maximize

$$\sum_{i,j} e_{ij} x_{ij} = 4 \cdot x_{00} + 5 \cdot x_{01} + 2 \cdot x_{02} + 1 \cdot x_{10} + 3 \cdot x_{11} + 1 \cdot x_{12}$$

example $L = 2, R = 3$

- subject to the constraints

example $L = 2, R = 3$

- subject to the constraints
- every lead gets assigned exactly 1 rep

$$x_{00} + x_{01} + x_{02} = 1, (l_0)$$
$$x_{10} + x_{11} + x_{12} = 1, (l_1)$$

example $L = 2, R = 3$

- subject to the constraints
- every lead gets assigned exactly 1 rep

$$x_{00} + x_{01} + x_{02} = 1, (l_0)$$
$$x_{10} + x_{11} + x_{12} = 1, (l_1)$$

- every rep visits at most 1 lead

$$x_{00} + x_{10} \leq 1, (r_0)$$
$$x_{01} + x_{11} \leq 1, (r_1)$$
$$x_{02} + x_{12} \leq 1, (r_2)$$

example $L = 2, R = 3$

- Given expected profits

$$(e_{ij}) = \begin{pmatrix} 4 & 5 & 2 \\ 1 & 3 & 1 \end{pmatrix}$$

example $L = 2, R = 3$

- Given expected profits

$$(e_{ij}) = \begin{pmatrix} 4 & 5 & 2 \\ 1 & 3 & 1 \end{pmatrix}$$

- $(x_{ij}) = ?$

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}}_{7}, \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{5}, \underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}}_{6},$$

$$\underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{6}, \underbrace{\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}}_{3}, \underbrace{\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}}_{5}$$

example $L = 2, R = 3$

- Given expected profits

$$(e_{ij}) = \begin{pmatrix} 4 & 5 & 2 \\ 1 & 3 & 1 \end{pmatrix}$$

- $(x_{ij}) = ?$

$$\underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}}_{7}, \underbrace{\begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{5}, \underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}}_{6},$$

$$\underbrace{\begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}}_{6}, \underbrace{\begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}}_{3}, \underbrace{\begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}}_{5}$$

- **upshot**: r_1 should not be greedy!

combinatorial optimization

combinatorial optimization

- difficult in general but can be done in practice

combinatorial optimization

- difficult in general but can be done in practice
- **integer linear programming**:
  - linear objective and constraints
  - integer decision variables

combinatorial optimization

- difficult in general but can be done in practice
- **integer linear programming**:
  - linear objective and constraints
  - integer decision variables
- **constraint programming**:
  - determine feasible solutions to a system of constraints
  - can be used to solve integer linear programs

example $L = 5, R = 7$

example $L = 5, R = 7$

- Given

$$(e_{ij}) = \begin{pmatrix} 7 & 4 & 8 & 5 & 7 & 3 & 7 \\ 8 & 5 & 4 & 8 & 8 & 3 & 6 \\ 5 & 2 & 8 & 6 & 2 & 5 & 1 \\ 6 & 9 & 1 & 3 & 7 & 4 & 9 \\ 3 & 5 & 3 & 7 & 5 & 9 & 7 \end{pmatrix}$$

example $L = 5, R = 7$

- Given

$$(e_{ij}) = \begin{pmatrix} 7 & 4 & 8 & 5 & 7 & 3 & 7 \\ 8 & 5 & 4 & 8 & 8 & 3 & 6 \\ 5 & 2 & 8 & 6 & 2 & 5 & 1 \\ 6 & 9 & 1 & 3 & 7 & 4 & 9 \\ 3 & 5 & 3 & 7 & 5 & 9 & 7 \end{pmatrix}$$

- find (x_{ij}) such that we maximum expected profit subject to the constraints:

example $L = 5, R = 7$

- Given

$$(e_{ij}) = \begin{pmatrix} 7 & 4 & 8 & 5 & 7 & 3 & 7 \\ 8 & 5 & 4 & 8 & 8 & 3 & 6 \\ 5 & 2 & 8 & 6 & 2 & 5 & 1 \\ 6 & 9 & 1 & 3 & 7 & 4 & 9 \\ 3 & 5 & 3 & 7 & 5 & 9 & 7 \end{pmatrix}$$

- find (x_{ij}) such that we maximum expected profit subject to the constraints:
  - every row of (x_{ij}) has exactly one 1

example $L = 5, R = 7$

- Given

$$(e_{ij}) = \begin{pmatrix} 7 & 4 & 8 & 5 & 7 & 3 & 7 \\ 8 & 5 & 4 & 8 & 8 & 3 & 6 \\ 5 & 2 & 8 & 6 & 2 & 5 & 1 \\ 6 & 9 & 1 & 3 & 7 & 4 & 9 \\ 3 & 5 & 3 & 7 & 5 & 9 & 7 \end{pmatrix}$$

- find (x_{ij}) such that we maximum expected profit subject to the constraints:
  - every row of (x_{ij}) has exactly one 1
  - every column of (x_{ij}) has at most one 1

CP-SAT example

CP-SAT example

```python
import numpy as np
from loguru import logger
from ortools.sat.python import cp_model
```

imports

CP-SAT example

```python
import numpy as np
from loguru import logger
from ortools.sat.python import cp_model
```

imports

```python
L = 5  # number of leads
R = 7  # number of reps
TIMEOUT_SECONDS = 5000
```

globals

CP-SAT example

```python
import numpy as np
from loguru import logger
from ortools.sat.python import cp_model
```

imports

```python
L = 5  # number of leads
R = 7  # number of reps
TIMEOUT_SECONDS = 5000
```

globals

```python
# random data
np.random.seed(42)
e = np.random.randint(1, 10, size=(L, R))  # e_ij = expected profit of assigning lead i to rep j
```

e_{ij}

CP-SAT example

```python
import numpy as np
from loguru import logger
from ortools.sat.python import cp_model
```

imports

```python
L = 5  # number of leads
R = 7  # number of reps
TIMEOUT_SECONDS = 5000
```

globals

```python
# random data
np.random.seed(42)
e = np.random.randint(1, 10, size=(L, R))  # e_ij = expected profit of assigning lead i to rep j
```

e_{ij}

```python
model = cp_model.CpModel()
```

CP-SAT example

```
# x_{ij}
x = [
    [model.NewBoolVar(f"x[{i},{j}]") for j in range(R)] for i in range(L)
]
```

x_{ij}

CP-SAT example

```
# x_{ij}
x = [
    [model.NewBoolVar(f"x[{i},{j}]") for j in range(R)] for i in range(L)
]
```

$$x_{ij}$$

```
# objective
objective_terms = [e[i][j] * x[i][j] for i in range(L) for j in range(R)]
model.Maximize(sum(objective_terms))
logger.info(f"objective: sum({objective_terms})")
```

$$\sum_{i,j} e_{ij} x_{ij}$$

CP-SAT example

```
# x_{ij}
x = [
    [model.NewBoolVar(f"x[{i},{j}]") for j in range(R)] for i in range(L)
]
```

$$x_{ij}$$

```
# objective
objective_terms = [e[i][j] * x[i][j] for i in range(L) for j in range(R)]
model.Maximize(sum(objective_terms))
logger.info(f"objective: sum({objective_terms})")
```

$$\sum_{i,j} e_{ij} x_{ij}$$

```
# constraint: sum_j x_{ij} = 1 : every lead is assigned to exactly one rep
for i in range(L):
    model.Add(sum(x[i][j] for j in range(R)) == 1)
    logger.info(f"constraint sum_j x_ij=1 ({i=}): sum_j x[{i},j] = 1")
```

$$\sum_{j=0}^{R-1} x_{ij} = 1$$

CP-SAT example

```python
# constraint: sum_i x_{ij} <= 1 : every rep visits at most one lead
for j in range(R):
    model.Add(sum(x[i][j] for i in range(L)) <= 1)
    logger.info(f"constraint sum_i x_ij<=1 ({j=}): sum_i x[i,{j}] <= 1")
```

$$\sum_{i=0}^{L-1} x_{ij} \leq 1$$

CP-SAT example

```python
# constraint: sum_i x_{ij} <= 1 : every rep visits at most one lead
for j in range(R):
    model.Add(sum(x[i][j] for i in range(L)) <= 1)
    logger.info(f"constraint sum_i x_ij<=1 ({j=}): sum_i x[i,{j}] <= 1")
```

$$\sum_{i=0}^{L-1} x_{ij} \leq 1$$

```python
# solve
solver = cp_model.CpSolver()
solver.parameters.max_time_in_seconds = TIMEOUT_SECONDS
logger.info(f"starting solver with {L=}, {R=}, {TIMEOUT_SECONDS=}")
status = solver.Solve(model)
```

solve

CP-SAT example

```python
# result
if status in [cp_model.OPTIMAL, cp_model.FEASIBLE]:
    logger.info(f"objective value: {solver.ObjectiveValue()=}")
    logger.info(f"status: {status=}, {solver.StatusName(status)}")
    for i in range(L):
        for j in range(R):
            if solver.Value(x[i][j]):
                logger.info(f"Lead {i} is assigned to Rep {j}")
else:
    logger.error("No solution found or time limit reached.")
```

result

CP-SAT example

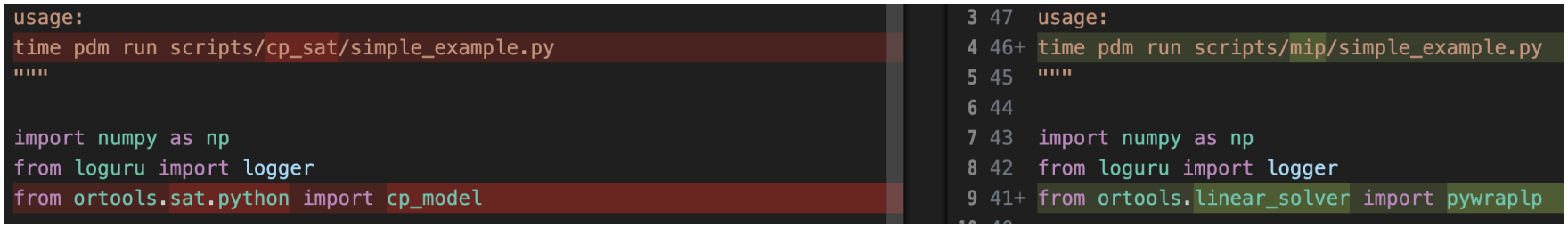
```python
# result
if status in [cp_model.OPTIMAL, cp_model.FEASIBLE]:
    logger.info(f"objective value: {solver.ObjectiveValue()=}")
    logger.info(f"status: {status=}, {solver.StatusName(status)}")
    for i in range(L):
        for j in range(R):
            if solver.Value(x[i][j]):
                logger.info(f"Lead {i} is assigned to Rep {j}")
else:
    logger.error("No solution found or time limit reached.")
```

result

- main/scripts/cp_sat/simple_example.py
- https://developers.google.com/optimization/cp

MIP example

MIP example



```
usage:
time pdm run scripts/cp_sat/simple_example.py
"""

import numpy as np
from loguru import logger
from ortools.sat.python import cp_model
```

```
3 47  usage:
4 46+ time pdm run scripts/mip/simple_example.py
5 45  """
6 44
7 43  import numpy as np
8 42  from loguru import logger
9 41+ from ortools.linear_solver import pywraplp
```

setup

MIP example

```
usage:
time pdm run scripts/cp_sat/simple_example.py
"""

import numpy as np
from loguru import logger
from ortools.sat.python import cp_model
```

```
usage:
time pdm run scripts/mip/simple_example.py
"""

import numpy as np
from loguru import logger
from ortools.linear_solver import pywraplp
```

setup

```
model = cp_model.CpModel()
```

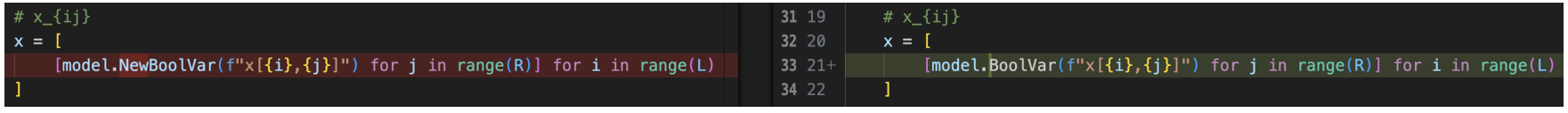
```
    model = pywraplp.Solver.CreateSolver("SCIP")
```

model

MIP example

setup

model

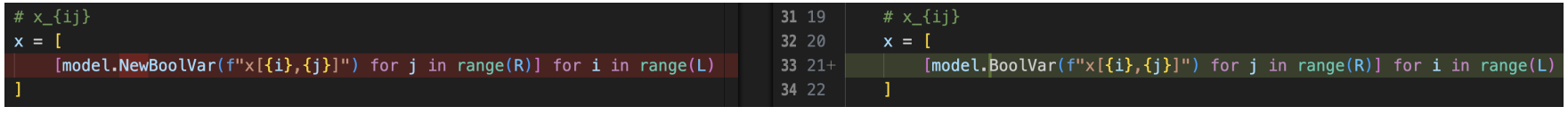


decision variables

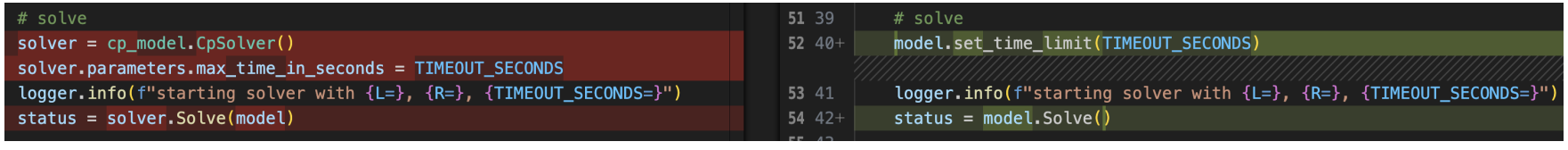
MIP example

setup

model

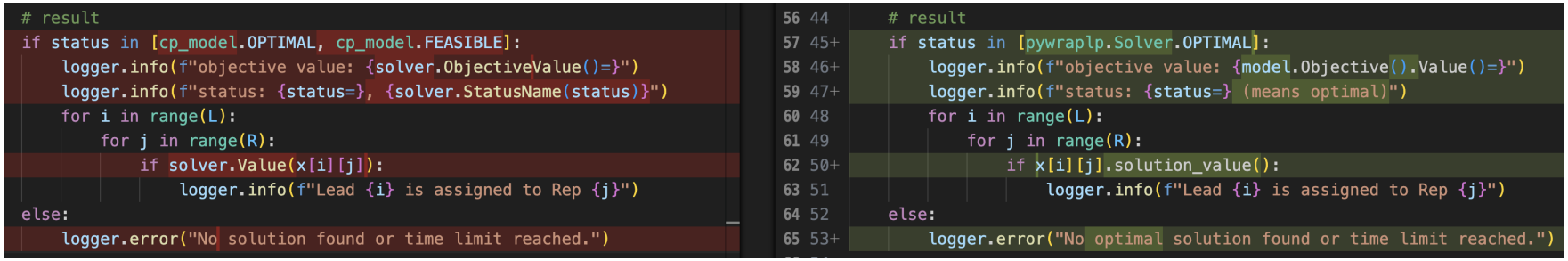


decision variables



solve

MIP example

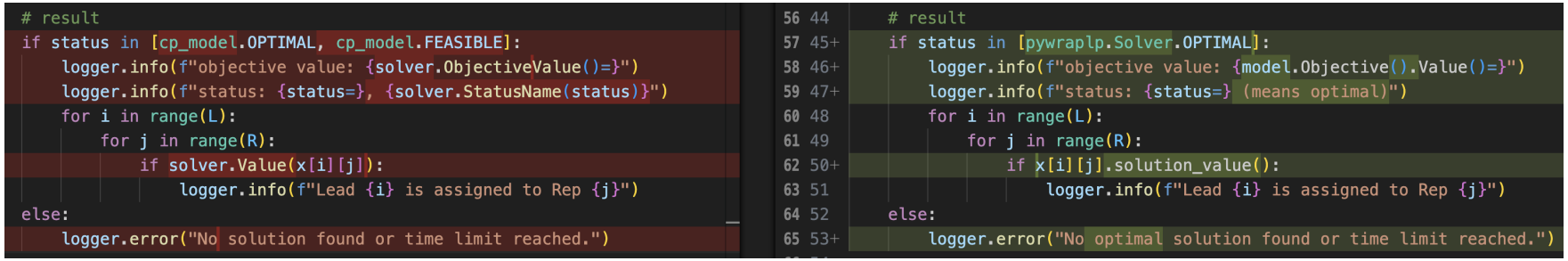


```
# result
if status in [cp_model.OPTIMAL, cp_model.FEASIBLE]:
    logger.info(f"objective value: {solver.ObjectiveValue()=}")
    logger.info(f"status: {status=}, {solver.StatusName(status)}")
    for i in range(L):
        for j in range(R):
            if solver.Value(x[i][j]):
                logger.info(f"Lead {i} is assigned to Rep {j}")
else:
    logger.error("No solution found or time limit reached.")
```

```
56 44    # result
57 45+   if status in [pywraplp.Solver.OPTIMAL]:
58 46+       logger.info(f"objective value: {model.Objective().Value()=}")
59 47+       logger.info(f"status: {status=} (means optimal)")
60 48        for i in range(L):
61 49            for j in range(R):
62 50+               if x[i][j].solution_value():
63 51                    logger.info(f"Lead {i} is assigned to Rep {j}")
64 52    else:
65 53+       logger.error("No optimal solution found or time limit reached.")
```

results

MIP example



```
# result
if status in [cp_model.OPTIMAL, cp_model.FEASIBLE]:
    logger.info(f"objective value: {solver.ObjectiveValue()=}")
    logger.info(f"status: {status=}, {solver.StatusName(status)}")
    for i in range(L):
        for j in range(R):
            if solver.Value(x[i][j]):
                logger.info(f"Lead {i} is assigned to Rep {j}")
else:
    logger.error("No solution found or time limit reached.")
```

```
56 44   # result
57 45+  if status in [pywraplp.Solver.OPTIMAL]:
58 46+      logger.info(f"objective value: {model.Objective().Value()=}")
59 47+      logger.info(f"status: {status=} (means optimal)")
60 48       for i in range(L):
61 49           for j in range(R):
62 50+              if x[i][j].solution_value():
63 51                   logger.info(f"Lead {i} is assigned to Rep {j}")
64 52   else:
65 53+      logger.error("No optimal solution found or time limit reached.")
```

results

- main/scripts/mip/simple_example.py
- https://developers.google.com/optimization/mip

solutions

```
objective value: solver.ObjectiveValue()=41.0
status: status=4, OPTIMAL
Lead 0 is assigned to Rep 6
Lead 1 is assigned to Rep 4
Lead 2 is assigned to Rep 2
Lead 3 is assigned to Rep 1
Lead 4 is assigned to Rep 5
```

```
objective value: model.Objective().Value()=41.0
status: status=0 (means optimal)
Lead 0 is assigned to Rep 0
Lead 1 is assigned to Rep 3
Lead 2 is assigned to Rep 2
Lead 3 is assigned to Rep 6
Lead 4 is assigned to Rep 5
```

closing

closing

- optimization problems pair well with predictions

closing

- optimization problems pair well with predictions
- optimal solutions are feasible

closing

- optimization problems pair well with predictions
- optimal solutions are feasible
- or-tools:
  - https://developers.google.com/optimization
  - https://github.com/d-krupke/cpsat-primer

closing

- optimization problems pair well with predictions
- optimal solutions are feasible
- or-tools:
  - https://developers.google.com/optimization
  - https://github.com/d-krupke/cpsat-primer
- https://github.com/faradayio/RepAssignment

closing

- optimization problems pair well with predictions
- optimal solutions are feasible
- or-tools:
  - https://developers.google.com/optimization
  - https://github.com/d-krupke/cpsat-primer
- https://github.com/faradayio/RepAssignment
- Thanks for listening!

setup with time slots

Now suppose that leads sign up for one of T daily time slots and that rep availability can vary

setup with time slots

Now suppose that leads sign up for one of T daily time slots and that rep availability can vary

$$
\begin{aligned}
l_i &= \text{lead } i, \quad i \in \{0, \ldots, L-1\} \\
r_j &= \text{rep } j, \quad j \in \{0, \ldots, R-1\} \\
t_k &= \text{time slot } k, \quad k \in \{0, \ldots, T-1\} \\
t_{s_i} &= \text{time slot for } l_i, \quad s_i \in \{0, \ldots, T-1\} \\
e_{ij} &= \text{expected profit of assigning } l_i \text{ to } r_j
\end{aligned}
$$

$$
x_{ij} = \begin{cases} 1 & \text{if } l_i \text{ is assigned to } r_j \\ 0 & \text{otherwise} \end{cases}
$$

$$
a_{jk} = \begin{cases} 1 & \text{if } r_j \text{ is available at } t_k \\ 0 & \text{otherwise} \end{cases}
$$

objective

We aim to maximize

$$\sum_{i,j} e_{ij} x_{ij}$$

subject to the following constraints

constraints

- For every l_i

$$\sum_{j=0}^{R-1} x_{ij} a_{js_i} = 1$$

constraints

- For every l_i

$$\sum_{j=0}^{R-1} x_{ij} a_{js_i} = 1$$

- Note that a_{js_i} is the availability of r_j at the time slot corresponding to l_i (since this time slot has index s_i)

constraints

- For every l_i

$$\sum_{j=0}^{R-1} x_{ij} a_{js_i} = 1$$

- Note that a_{js_i} is the availability of r_j at the time slot corresponding to l_i (since this time slot has index s_i)
- i.e. *every lead must be assigned exactly 1 available rep*

constraints

- For every r_j and every t_k

$$\sum_{i=0}^{L-1} x_{ij} a_{jk} \leq 1$$

constraints

- For every r_j and every t_k

$$\sum_{i=0}^{L-1} x_{ij} a_{jk} \leq 1$$

- i.e. *a rep cannot have more than 1 assignment in each time slot*

constraints

- For every r_j and every t_k

$$\sum_{i=0}^{L-1} x_{ij} a_{jk} \leq 1$$

- i.e. *a rep cannot have more than 1 assignment in each time slot*
- Note that for a given j we do allow

$$\sum_{i=0}^{L-1} x_{ij} a_{js_i} \geq 1$$